

Polars Cheat Sheet

23 tables · 335 concepts · Updated 2026-05-28 · cheatgrid.com

Polars is a blazingly fast DataFrame library built in Rust and designed for high-performance data processing. Its multi-threaded query engine uses Apache Arrow columnar memory, supports both eager and lazy execution, and since Polars 1.0 offers a production-stable API. The new streaming engine (2025–2026) processes datasets larger than RAM in batches, optional GPU acceleration via NVIDIA RAPIDS runs queries on CUDA hardware, and Polars Cloud brings distributed execution to any scale. This cheat sheet covers everything from basic DataFrame operations to advanced optimization, streaming sinks, GPU execution, and the latest type system additions including stable [Decimal](#) , [Int128](#) , [Enum](#) , and [Array](#) .

CONTENTS

Fundamentals	2
Expressions and Contexts	3
Lazy vs Eager Execution	3
Query Optimization	4
Data Selection and Filtering	5
Joins	5
Aggregations and Group By	6
Window Functions	7
File I/O and Scanning	8
Schema Handling	9
Streaming and Sinks	10
Performance Tuning	10
Interoperability	11
String Operations	12
Datetime Operations	12
List Operations	13
Null Handling	14
Rolling and Time-Series Aggregations	14
Pivoting and Reshaping	15
Advanced Operations	16
Statistical Functions	16
Column Selectors	17
Practical Patterns and Examples	18
References	19

TABLE 1

Fundamentals

The entry points to Polars — creating DataFrames, reading common file formats, and inspecting data. These methods cover the most frequent first steps in any Polars script or notebook.

Method	Example	Description
Import Polars	<code>import polars as pl</code>	- Standard import convention <code>pl</code> is the universal alias
Create DataFrame	<code>df = pl.DataFrame({"col1": [1, 2], "col2": ["a", "b"]})</code>	Create DataFrame from dict, list of dicts, or Series
Read CSV	<code>df = pl.read_csv("data.csv")</code>	Read CSV into an eager DataFrame with full data in memory
Read Parquet	<code>df = pl.read_parquet("data.parquet")</code>	- Read Parquet into memory - preferred format for performance
LazyFrame	<code>lf = pl.scan_csv("data.csv")</code>	- Create a lazy query plan without loading data - enables optimization
Collect Lazy	<code>df = lf.collect()</code>	Execute lazy plan and materialize results into a DataFrame
Head	<code>df.head(10)</code>	Return first n rows for quick inspection
Tail	<code>df.tail(10)</code>	Return last n rows
Shape	<code>df.shape</code>	Returns <code>(rows, columns)</code> tuple
Columns	<code>df.columns</code>	List all column names
Schema	<code>df.schema</code>	Mapping of column names to data types
Dtypes	<code>df.dtypes</code>	List data types for all columns in order
Describe	<code>df.describe()</code>	Summary statistics (count, mean, std, min, max, etc.)
Glimpse	<code>df.glimpse()</code>	- One-line-per-column overview of schema and sample values - useful for wide DataFrames
Write CSV	<code>df.write_csv("output.csv")</code>	Export DataFrame to CSV
Write Parquet	<code>df.write_parquet("output.parquet")</code>	- Export to Parquet with compression - best format for most data
Value Counts	<code>df["city"].value_counts(sort=True)</code>	Count occurrences of each unique value in a Series
Read Excel	<code>df = pl.read_excel("data.xlsx", sheet_name="Sheet1")</code>	Read Excel file using the fast calamine engine by default
Write Excel	<code>df.write_excel("output.xlsx", worksheet="data")</code>	Write DataFrame to an Excel file (requires <code>xlsxwriter</code>)
Read Database	<code>df = pl.read_database("SELECT * FROM tbl", conn)</code>	Read SQL query results from any database connection
Read Database URI	<code>df = pl.read_database_uri("SELECT * FROM tbl", "postgresql://user:pw@host/db")</code>	Read from database using a URI connection string (connectorx or ADBC)

TABLE 2

Expressions and Contexts

Expressions are the building blocks of all Polars queries — reusable, composable descriptions of transformations that the engine evaluates in parallel. Understanding the four contexts (`select`, `with_columns`, `filter`, `group_by`) is the key mental model for writing idiomatic Polars.

Method	Example	Description
pl.col	<code>pl.col("age")</code>	- Reference a column by name - the foundation of all expressions
select	<code>df.select(pl.col("name"), pl.col("age"))</code>	- Select and/or transform columns - returns new DataFrame
with_columns	<code>df.with_columns(pl.col("age") + 1)</code>	Add or replace columns while keeping all others
filter	<code>df.filter(pl.col("age") > 25)</code>	Keep rows where boolean expression is true
group_by	<code>df.group_by("city").agg(pl.col("age").mean())</code>	Group rows and apply aggregation expressions
alias	<code>pl.col("age").alias("years")</code>	Rename expression result to a new column name
when / then / otherwise	<code>pl.when(pl.col("age") > 18).then(pl.lit("adult")).otherwise(pl.lit("minor"))</code>	Vectorized conditional — equivalent to SQL <code>CASE WHEN</code>
replace	<code>pl.col("code").replace({1: "a", 2: "b"})</code>	- Map specific values to new values - cleaner than <code>when/then</code> for lookups
cast	<code>pl.col("age").cast(pl.Float64)</code>	Convert column to a different data type
pl.lit	<code>pl.lit(10)</code>	Create a scalar literal expression for use in arithmetic or conditions
pl.len	<code>df.select(pl.len())</code>	- Count rows in context — equivalent to <code>COUNT(*)</code> in SQL - replaces <code>pl.count()</code>
pl.all()	<code>df.select(pl.all())</code>	- Select all columns - often used in aggregation (<code>pl.all().sum()</code>)
exclude	<code>df.select(pl.all().exclude("id"))</code>	Select all columns except specified ones
expression chaining	<code>pl.col("name").str.to_uppercase().str.strip_chars()</code>	- Chain multiple transformations - the engine optimizes the full chain
multiple aggregations	<code>df.select(pl.col("age").mean(), pl.col("salary").sum())</code>	Compute multiple aggregations in one pass
pl.int_range	<code>pl.int_range(0, pl.len(), dtype=pl.UInt32).alias("idx")</code>	- Generate integer sequence - use with <code>pl.len()</code> for row indices
col by regex	<code>df.select(pl.col("^*_id\$"))</code>	Select columns matching a regex pattern via <code>pl.col()</code>
col by dtype	<code>df.select(pl.col(pl.Int64))</code>	Select all columns of a specific data type

TABLE 3

Lazy vs Eager Execution

Lazy execution is the idiomatic Polars pattern for any non-trivial query. The engine builds a logical plan, applies rewrites (predicate pushdown, projection pruning, CSE), then executes optimally. The new `engine` parameter on `collect()` enables streaming or GPU execution without changing query syntax.

Method	Example	Description
scan_csv (lazy)	<code>lf = pl.scan_csv("data.csv")</code>	- Lazy CSV scan - does not load data until <code>.collect()</code>

Method	Example	Description
scan_parquet (lazy)	<code>lf = pl.scan_parquet("data.parquet")</code>	Lazy Parquet scan with predicate and projection pushdown
scan_ndjson (lazy)	<code>lf = pl.scan_ndjson("data.ndjson")</code>	Lazy scan for newline-delimited JSON
scan_ipc (lazy)	<code>lf = pl.scan_ipc("data.ipc")</code>	Lazy scan for Apache Arrow IPC/Feather format
lazy() convert	<code>lf = df.lazy()</code>	Convert an eager DataFrame to a LazyFrame
collect()	<code>df = lf.collect()</code>	Execute the lazy plan with all optimizations applied
collect engine=streaming	<code>df = lf.collect(engine="streaming")</code>	• Execute in the new streaming engine • processes data in batches for larger-than-RAM queries
collect engine=gpu	<code>df = lf.collect(engine="gpu")</code>	• Execute on an NVIDIA GPU via RAPIDS cuDF • requires <code>pip install polars[gpu]</code>
collect_batches	<code>for batch in lf.collect_batches(chunk_size=50_000): process(batch)</code>	• Streaming generator yielding sub-DataFrames • use when logic cannot be expressed as a sink
collect_all	<code>r1, r2 = pl.collect_all([lf1, lf2])</code>	Collect multiple LazyFrames with shared CSE optimization
profile()	<code>df, timings = lf.profile()</code>	Execute query and return a timing breakdown per node for performance investigation
explain()	<code>print(lf.explain())</code>	Print the optimized query plan without executing it
show_graph()	<code>lf.show_graph()</code>	Visualize query plan as a graph
fetch()	<code>df = lf.fetch(n_rows=100)</code>	• Execute on first n rows only • useful for schema checking during development
cache()	<code>lf_cached = lf.cache()</code>	Cache expensive intermediate results when a LazyFrame is reused in multiple branches

TABLE 4

Query Optimization

Polars applies these optimizations automatically on every lazy query. Understanding them helps you write queries that cooperate with the optimizer rather than fighting it. Most optimizations are on by default; you can inspect which applied using `explain()`.

Technique	Example	Description
Predicate Pushdown	<code>lf.filter(pl.col("age") > 25).collect()</code>	Pushes filter conditions down to the source so fewer rows are read
Projection Pushdown	<code>lf.select("name", "age").collect()</code>	Reads only required columns, reducing I/O and memory
Common Subplan Elimination	<code>pl.collect_all([lf.agg1(), lf.agg2()])</code>	Detects shared subplans across multiple LazyFrames and executes them once
Common Subexpr Elimination	<code>lf.with_columns(expr1, expr2)</code> where <code>expr1</code> and <code>expr2</code> share sub-expressions	Caches identical sub-expressions within a single query to avoid recomputation
Slice Pushdown	<code>lf.head(100).collect()</code>	Limits rows read from source when <code>.head()</code> , <code>.tail()</code> , or <code>.slice()</code> is used
Collapse Joins	<code>lf.join(other, ...).filter(...)</code>	Merges a join and adjacent filters into a single faster join operation
Cluster With Columns	Multiple sequential <code>with_columns(...)</code> calls	Combines independent sequential <code>with_columns</code> calls into a single pass

Technique	Example	Description
cache() Intermediate	<code>lf_cached = lf.cache()</code>	Manually cache an expensive result that is consumed more than once
engine affinity	<code>pl.Config.set_engine_affinity("streaming")</code>	Globally set default execution engine for all subsequent queries
Rechunk	<code>df.rechunk()</code>	Consolidate fragmented memory chunks for better cache locality after incremental builds

TABLE 5

Data Selection and Filtering

Core row and column selection patterns. These are the operations used most frequently in any Polars workflow — master them before moving to joins and aggregations.

Method	Example	Description
Filter rows	<code>df.filter(pl.col("age") > 30)</code>	Keep rows matching a boolean expression
AND filter	<code>df.filter((pl.col("age") > 25) & (pl.col("city") == "NYC"))</code>	Combine conditions with <code>&</code> (AND)
OR filter	<code>df.filter((pl.col("age") < 20) (pl.col("age") > 60))</code>	• Combine conditions with <code> </code> (OR)
is_in	<code>df.filter(pl.col("city").is_in(["NYC", "LA", "SF"]))</code>	Check if values are in a list
is_between	<code>df.filter(pl.col("age").is_between(20, 30))</code>	Check if values fall within a range (inclusive by default)
Sort	<code>df.sort("age", descending=True)</code>	Sort rows by one or more columns
Sort multiple	<code>df.sort(["city", "age"], descending=[False, True])</code>	Sort by multiple columns with per-column order direction
with_row_index	<code>df.with_row_index("idx")</code>	Add a zero-based integer row index column
Sample	<code>df.sample(n=100, seed=42)</code>	Randomly sample rows with optional reproducibility seed
Slice	<code>df.slice(10, 20)</code>	Extract rows with offset and length
Unique	<code>df.unique(subset=["name", "city"])</code>	Remove duplicate rows based on a subset of columns
Drop Nulls	<code>df.drop_nulls(subset=["age"])</code>	Remove rows with null values in specified columns
gather_every	<code>lf.gather_every(n=2, offset=0).collect()</code>	• Take every nth row • works on both DataFrame and LazyFrame
Select regex	<code>df.select(pl.col("^.*_id\$"))</code>	Select columns matching a regex pattern
Select by dtype	<code>df.select(pl.col(pl.Int64))</code>	Select all columns of a specific data type

TABLE 6

Joins

Polars joins are highly optimized in both the in-memory and streaming engines. The `how="full"` outer join (formerly `"outer"`) and the new `join_where()` for inequality predicates cover the most common relational join patterns.

Method	Example	Description
Inner Join	<code>df1.join(df2, on="id", how="inner")</code>	Keep only rows with matching keys in both DataFrames
Left Join	<code>df1.join(df2, on="id", how="left")</code>	Keep all rows from left DataFrame, add matching from right

Method	Example	Description
Right Join	<code>df1.join(df2, on="id", how="right")</code>	Keep all rows from right DataFrame, add matching from left
Full Outer Join	<code>df1.join(df2, on="id", how="full", coalesce=True)</code>	- Keep all rows from both - nulls where no match (renamed from "outer" in 1.0)
Semi Join	<code>df1.join(df2, on="id", how="semi")</code>	- Keep rows from left that have at least one match in right - no right columns returned
Anti Join	<code>df1.join(df2, on="id", how="anti")</code>	Keep rows from left that have no match in right
Cross Join	<code>df1.join(df2, how="cross")</code>	Cartesian product — all combinations of rows from both
join_where (inequality)	<code>lf1.join_where(lf2, pl.col("dur") < pl.col("time"))</code>	- Inner join on one or more inequality predicates - supports <code><</code>, <code>></code>, <code><=</code>, <code>>=</code>
AsOf Join	<code>df1.join_asof(df2, on="timestamp", strategy="backward")</code>	- Join on nearest key match - ideal for time-series alignment
Join multiple keys	<code>df1.join(df2, on=["city", "state"], how="inner")</code>	Join on multiple columns simultaneously
Join different names	<code>df1.join(df2, left_on="id", right_on="user_id")</code>	Join when key column names differ
Join validate	<code>df1.join(df2, on="id", validate="m:1")</code>	Assert join cardinality: <code>'1:1'</code> , <code>'1:m'</code> , <code>'m:1'</code> , <code>'m:m'</code>
nulls_equal	<code>df1.join(df2, on="id", nulls_equal=True)</code>	Treat nulls as equal join keys (renamed from <code>join_nulls</code> in 1.24)
Join suffix	<code>df1.join(df2, on="id", suffix="_right")</code>	Suffix appended to duplicate column names from right DataFrame
Concat vertical	<code>pl.concat([df1, df2], how="vertical")</code>	Stack DataFrames row-wise (union, same schema)
Concat horizontal	<code>pl.concat([df1, df2], how="horizontal")</code>	Concatenate DataFrames side-by-side (same row count)

TABLE 7

Aggregations and Group By

Group-by and aggregation are where Polars' parallel execution shines most. Multiple aggregations in one `.agg()` call execute in parallel across groups — always batch your aggregations rather than chaining multiple `group_by` calls.

Method	Example	Description
group_by single	<code>df.group_by("city").agg(pl.col("age").mean())</code>	Group by one column and apply aggregation
group_by multiple	<code>df.group_by(["city", "state"]).agg(pl.col("age").mean())</code>	Group by multiple columns for hierarchical aggregation
Multiple agg	<code>df.group_by("city").agg(pl.col("age").mean(), pl.col("salary").sum())</code>	- Multiple aggregations in one pass - always prefer this over chaining
maintain_order	<code>df.group_by("city", maintain_order=True).agg(...)</code>	Preserve input row order in results (slight performance cost)
pl.len (group count)	<code>df.group_by("city").agg(pl.len().alias("count"))</code>	- Count rows per group - replaces deprecated <code>pl.count()</code>
sum	<code>pl.col("salary").sum()</code>	Sum all non-null values
mean	<code>pl.col("age").mean()</code>	Arithmetic mean
median	<code>pl.col("age").median()</code>	Median (50th percentile)

Method	Example	Description
min / max	<code>pl.col("age").min() , pl.col("age").max()</code>	Minimum and maximum values
std / var	<code>pl.col("age").std() , pl.col("age").var()</code>	Sample standard deviation and variance
first / last	<code>pl.col("name").first() , pl.col("name").last()</code>	First or last value in group
n_unique	<code>pl.col("city").n_unique()</code>	Count distinct values
list agg	<code>pl.col("name").implode()</code>	Collect all values in group into a List
quantile	<code>pl.col("age").quantile(0.75)</code>	Specific quantile (e.g., 75th percentile)
mode	<code>pl.col("category").mode()</code>	• Most frequent value • may return multiple values on tie
agg filter	<code>pl.col("price").filter(pl.col("status") == "sold").mean()</code>	Apply filter inside aggregation for conditional statistics

TABLE 8

Window Functions

Window functions compute a value for each row using surrounding rows defined by a partition, without collapsing the DataFrame to one row per group. The `.over()` expression is Polars' equivalent of SQL `PARTITION BY`.

Method	Example	Description
over() partition	<code>pl.col("salary").mean().over("department")</code>	• Compute group aggregate and broadcast back to each row • no collapse
over() multiple keys	<code>pl.col("salary").rank().over(["dept", "year"])</code>	Partition by multiple columns
row number	<code>pl.int_range(pl.len()).over("group").alias("row_num")</code>	Sequential row number within partition using <code>int_range</code>
rank	<code>pl.col("score").rank().over("category")</code>	• Rank within partition • handles ties via <code>method</code> parameter
cumulative sum	<code>pl.col("sales").cum_sum().over("month")</code>	Running cumulative sum within each partition
cumulative count	<code>pl.col("*").cum_count().over("group")</code>	Running count of non-null rows within partition
cumulative min / max	<code>pl.col("price").cum_min() , pl.col("price").cum_max()</code>	Running minimum and maximum
shift (lag)	<code>pl.col("price").shift(1).over("stock")</code>	Access previous row values within partition
shift (lead)	<code>pl.col("price").shift(-1).over("stock")</code>	Access next row values (negative n = lead)
diff	<code>pl.col("price").diff().over("stock")</code>	Difference from previous row value
pct_change	<code>pl.col("price").pct_change().over("stock")</code>	Percentage change from previous row
rolling window	<code>pl.col("price").rolling_mean(window_size=3).over("stock")</code>	Sliding window mean within partition
forward fill	<code>pl.col("value").forward_fill().over("group")</code>	Propagate last valid value forward within partition
backward fill	<code>pl.col("value").backward_fill().over("group")</code>	Propagate next valid value backward within partition

TABLE 9

File I/O and Scanning

Polars supports all major file formats with both eager (`read_*` / `write_*`) and lazy (`scan_*` / `sink_*`) interfaces. Prefer `scan_*` + `sink_*` for large-file workflows — they enable full query optimization and streaming I/O without materializing intermediate results in RAM.

Method	Example	Description
scan_csv options	<code>pl.scan_csv("data.csv", separator=";", has_header=True)</code>	Lazy CSV scan with delimiter and header options
scan_parquet glob	<code>pl.scan_parquet("data/*.parquet")</code>	Scan multiple Parquet files matching a glob pattern
scan_ndjson	<code>lf = pl.scan_ndjson("data.ndjson")</code>	Lazy scan for newline-delimited JSON
scan_ipc	<code>lf = pl.scan_ipc("data.feather")</code>	Lazy scan for Apache Arrow IPC/Feather format
sink_parquet	<code>lf.sink_parquet("out.parquet")</code>	- Stream lazy query results directly to Parquet - no full collect needed
sink_csv	<code>lf.sink_csv("out.csv")</code>	Stream lazy query results directly to CSV
sink_ndjson	<code>lf.sink_ndjson("out.ndjson")</code>	Stream lazy query results to newline-delimited JSON
sink_ipc	<code>lf.sink_ipc("out.feather")</code>	Stream lazy query results to Arrow IPC format
PartitionBy	<code>lf.sink_parquet(pl.PartitionBy("./out/", key="year"), mkdir=True)</code>	Write hive-partitioned Parquet using the <code>PartitionBy</code> API
CSV null values	<code>pl.read_csv("data.csv", null_values=["NA", "NULL", ""])</code>	Interpret specific strings as null when reading CSV
CSV skip rows	<code>pl.read_csv("data.csv", skip_rows=5)</code>	Skip header rows before parsing
schema override (read)	<code>pl.read_csv("data.csv", schema_overrides={"age": pl.Int64})</code>	Override inferred column types when reading
Parquet row groups	<code>df.write_parquet("data.parquet", row_group_size=100_000)</code>	Control row group size for query performance optimization
read JSON	<code>df = pl.read_json("data.json")</code>	Read JSON array file into DataFrame
read NDJSON	<code>df = pl.read_ndjson("data.ndjson")</code>	Read newline-delimited JSON (streaming-friendly format)
read IPC / Feather	<code>df = pl.read_ipc("data.feather")</code>	Read Arrow IPC (Feather v2) format — fastest serialization format
read Excel	<code>df = pl.read_excel("data.xlsx", sheet_name="Sales")</code>	Read Excel into DataFrame using the fast Calamine engine by default
write Excel	<code>df.write_excel("report.xlsx", worksheet="Summary")</code>	Write to Excel with optional multi-sheet and formatting support
read database	<code>df = pl.read_database("SELECT * FROM orders", conn)</code>	Read SQL query results from any DBAPI2, SQLAlchemy, or ODBC connection
read database URI	<code>df = pl.read_database_uri("SELECT * FROM orders", "postgresql://user:pw@host/db")</code>	Faster than <code>read_database</code> for large tables via connectorx or ADBC
Hugging Face datasets	<code>pl.scan_parquet("hf://datasets/username/dataset/**")</code>	Scan Hugging Face datasets directly using <code>hf://</code> URI

TABLE 10

Schema Handling

Polars enforces a strict, known schema at all times. Getting types right at the boundary (reading, casting, or constructing) pays dividends in performance and correctness throughout your pipeline.

Type	Example	Description
schema dict	<code>pl.DataFrame(data, schema={"id": pl.Int64, "name": pl.String})</code>	Explicitly define column names and types at construction
schema_overrides	<code>pl.read_csv("data.csv", schema_overrides={"age": pl.Int64})</code>	Override specific inferred types when reading files
cast	<code>df.with_columns(pl.col("age").cast(pl.Int32))</code>	Convert column to a different data type
cast strict=False	<code>pl.col("age").cast(pl.Int64, strict=False)</code>	Allow cast failures to produce nulls instead of errors
rename	<code>df.rename({"old_name": "new_name"})</code>	Rename one or more columns
drop	<code>df.drop("col1", "col2")</code>	Remove specified columns
String type (pl.String)	<code>pl.col("name").cast(pl.String)</code>	- UTF-8 string type <code>pl.Utf8</code> is a backward-compatible alias
Integer types	<code>pl.Int8, pl.Int16, pl.Int32, pl.Int64, pl.Int128</code>	- Signed integers <code>Int128</code> supports values up to $\pm 1.7e38$
Unsigned integer types	<code>pl.UInt8, pl.UInt16, pl.UInt32, pl.UInt64</code>	- Unsigned integers <code>UInt8</code> is ideal for 0–255 encoded categoricals
Float types	<code>pl.Float32, pl.Float64</code>	- IEEE 754 floating point <code>Float16</code> also available
Decimal type	<code>pl.Decimal(precision=9, scale=2)</code>	Stable 128-bit exact decimal - use for financial data where float rounding is unacceptable
Categorical type	<code>pl.col("category").cast(pl.Categorical)</code>	- Dictionary-encoded strings with categories inferred at runtime - efficient for low-cardinality columns
Enum type	<code>pl.col("priority").cast(pl.Enum(["High", "Medium", "Low"]))</code>	Ordered categorical with a fixed, predefined set of values - more performant than <code>Categorical</code> when categories are known upfront
Struct type	<code>pl.col("nested").struct.field("subfield")</code>	Nested data structure with named fields
List type	<code>pl.col("tags").list.len()</code>	Variable-length list of homogeneous values per row
Array type	<code>pl.Series("a", [[1,2],[3,4]], dtype=pl.Array(pl.Int64, 2))</code>	Fixed-length array - more efficient than <code>List</code> when all rows have the same length
Temporal types	<code>pl.Date, pl.Datetime, pl.Time, pl.Duration</code>	Date, timestamp, time-of-day, and duration types
null_count	<code>pl.col("age").null_count()</code>	Count null values in a column

TABLE 11

Streaming and Sinks

The new streaming engine (Polars 1.x, 2025) processes queries in morsel-driven batches, enabling larger-than-RAM workloads. The `sink_*` methods are the fastest way to write streaming results directly to disk without materializing a full `DataFrame` in memory.

Method	Example	Description
<code>collect(engine="streaming")</code>	<code>lf.collect(engine="streaming")</code>	- Execute query in streaming engine - batched execution with automatic CPU fallback
<code>sink_parquet</code>	<code>lf.sink_parquet("out.parquet")</code>	Stream query results to Parquet without holding full result in memory
<code>sink_csv</code>	<code>lf.sink_csv("out.csv")</code>	Stream query results to CSV
<code>sink_ndjson</code>	<code>lf.sink_ndjson("out.ndjson")</code>	Stream query results to newline-delimited JSON
<code>sink_ipc</code>	<code>lf.sink_ipc("out.feather")</code>	Stream query results to Arrow IPC format
PartitionBy sink	<code>lf.sink_parquet(pl.PartitionBy("./out/", key="year"), mkdir=True)</code>	Write hive-partitioned output in a single streaming pass
<code>collect_batches</code>	<code>for batch in lf.collect_batches(chunk_size=50_000): process(batch)</code>	- Generator of <code>DataFrames</code> - use only when custom per-batch Python logic is needed
set engine affinity	<code>pl.Config.set_engine_affinity("streaming")</code>	Set streaming as default engine for all subsequent queries in the session
streaming large filter	<code>pl.scan_parquet("huge.parquet").filter(...).sink_parquet("filtered.parquet")</code>	End-to-end streaming pipeline: scan → filter → sink without RAM overflow
streaming group by	<code>lf.group_by("key").agg(...).collect(engine="streaming")</code>	Group-by aggregation in streaming mode
<code>collect_all CSE</code>	<code>pl.collect_all([lf1, lf2])</code>	- Collect multiple <code>LazyFrames</code> in one call - shared subplans execute only once

TABLE 12

Performance Tuning

Polars is fast by default, but these patterns maximize throughput and minimize memory for production workloads. The most impactful wins are: use lazy API with sinks, avoid intermediate collects, use Categorical for string columns, and target the streaming or GPU engine for bottleneck queries.

Technique	Example	Description
GPU engine	<code>lf.collect(engine="gpu")</code>	- Execute on NVIDIA GPU via RAPIDS cuDF - install with <code>pip install polars[gpu]</code> - auto-fallback to CPU if unsupported
GPU engine object	<code>lf.collect(engine=pl.GPUEngine(device=0))</code>	- Target a specific GPU device <code>raise_on_fail=True</code> disables CPU fallback
Lazy for large data	<code>pl.scan_parquet("large.parquet").filter(...).sink_parquet("out.parquet")</code>	Use lazy API with sinks for large data — avoids loading entire result into RAM
Avoid intermediate collects	Build full <code>lf.filter().select().group_by()...</code> chain before calling <code>.collect()</code>	- Each <code>.collect()</code> breaks optimization boundaries - chain operations before materializing
Categorical for strings	<code>df.with_columns(pl.col("category").cast(pl.Categorical))</code>	- Encodes low-cardinality string columns as integers - reduces memory and speeds joins/group_by

Technique	Example	Description
Enum for known categories	<code>df.with_columns(pl.col("status").cast(pl.Enum(["open", "closed"])))</code>	More performant than <code>Categorical</code> when the set of values is fixed and known upfront
Filter early	<code>pl.scan_parquet("data.parquet").filter(...).select(...).collect()</code>	Even if the optimizer does pushdown, explicit early filtering makes intent clear
Projection early	<code>pl.scan_csv("data.csv").select("col1", "col2").collect()</code>	- Select only needed columns early - optimizer prunes at source
collect_all	<code>pl.collect_all([lf1, lf2, lf3])</code>	- Process multiple LazyFrames together - CSE avoids redundant source reads
Parallel execution	Automatic across all available CPU cores	- Polars uses a Rust thread pool - no configuration needed
rechunk	<code>df = df.rechunk()</code>	Consolidate fragmented Arrow buffers into contiguous memory for better cache performance
estimated_size	<code>df.estimated_size(unit="mb")</code>	Estimate RAM footprint of a DataFrame in bytes, KB, MB, or GB
Partitioned writes	<code>lf.sink_parquet(pl.PartitionBy("./data/", key="year"))</code>	Partition output by column for faster filtered reads on subsequent queries
String cache	<code>pl.enable_string_cache()</code>	- Enable global string cache - needed when joining on Categorical columns from different DataFrames

TABLE 13

Interoperability

Polars uses Apache Arrow as its internal memory format, enabling zero-copy data exchange with the broader Arrow ecosystem. Converting to/from pandas adds a copy overhead but is well-supported.

Method	Example	Description
From pandas	<code>df = pl.from_pandas(pandas_df)</code>	- Convert pandas DataFrame to Polars via Arrow - copies data
To pandas	<code>pandas_df = df.to_pandas()</code>	- Convert to pandas DataFrame - copies data
From Arrow	<code>df = pl.from_arrow(arrow_table)</code>	Create Polars DataFrame from PyArrow Table — zero-copy
To Arrow	<code>arrow_table = df.to_arrow()</code>	Convert to PyArrow Table — zero-copy
From NumPy	<code>df = pl.from_numpy(np_array, schema=["col1", "col2"])</code>	Create DataFrame from NumPy array
To NumPy	<code>np_array = df.to_numpy()</code>	- Convert DataFrame to NumPy array - copies data
From dict	<code>df = pl.DataFrame({"a": [1, 2], "b": [3, 4]})</code>	Create from Python dictionary
To dict	<code>data = df.to_dict(as_series=False)</code>	Convert to Python dictionary of lists
From records	<code>df = pl.from_records([{"a": 1, "b": 2}, {"a": 3, "b": 4}])</code>	Create from list of row dicts
Read database	<code>df = pl.read_database("SELECT * FROM tbl", conn)</code>	Read from DBAPI2, SQLAlchemy, ODBC, ADBC, or async drivers
Hugging Face	<code>pl.scan_parquet("hf://datasets/user/dataset/**")</code>	Lazy scan Hugging Face datasets using the <code>hf://</code> URI scheme
Arrow memory format	Polars uses Apache Arrow columnar format internally	Zero-copy sharing with DuckDB, Apache Arrow Flight, PyArrow, and cuDF

TABLE 14

String Operations

All string operations live under the `.str` namespace and are vectorized over the full column. They work in all contexts (select, with_columns, filter) and compose with other expressions.

Method	Example	Description
contains	<code>pl.col("text").str.contains("pattern")</code>	Check if string contains a substring or regex pattern
starts_with / ends_with	<code>pl.col("text").str.starts_with("prefix")</code> <code>, .str.ends_with("suffix")</code>	Check string prefix or suffix
to_uppercase / to_lowercase	<code>pl.col("name").str.to_uppercase()</code>	Convert string case
strip_chars	<code>pl.col("text").str.strip_chars()</code>	Remove leading and trailing whitespace (or specified characters)
replace	<code>pl.col("text").str.replace("old", "new")</code>	Replace first occurrence of pattern
replace_all	<code>pl.col("text").str.replace_all("old", "new")</code>	Replace all occurrences of pattern
replace_many	<code>pl.col("text").str.replace_many(["a", "b"], ["A", "B"])</code>	Replace multiple patterns in one pass using Aho-Corasick algorithm
split	<code>pl.col("text").str.split(",")</code>	Split string into a List by delimiter
extract	<code>pl.col("text").str.extract(r"(\d+)", group_index=1)</code>	Extract first regex match capture group
extract_all	<code>pl.col("text").str.extract_all(r"\d+")</code>	Extract all regex matches as a List
len_chars	<code>pl.col("text").str.len_chars()</code>	• Character count (Unicode-aware) • use <code>len_bytes()</code> for byte length
slice	<code>pl.col("text").str.slice(0, 5)</code>	Extract substring by start position and length
concat_str	<code>pl.concat_str([pl.col("first"), pl.col("last")], separator=" ")</code>	Concatenate multiple columns into one string with separator
strptime	<code>pl.col("date_str").str.strptime(pl.Date, "%Y-%m-%d")</code>	Parse string to date/datetime using a format string
to_integer	<code>pl.col("num_str").str.to_integer(base=10)</code>	Parse numeric strings to integer type

TABLE 15

Datetime Operations

All datetime operations live under the `.dt` namespace. Polars supports `Date`, `Datetime`, `Time`, and `Duration` types with full timezone support. Calendar-aware arithmetic (`"1mo"`, `"1y"`) correctly handles month-boundary and DST cases.

Method	Example	Description
strptime (parse)	<code>pl.col("date_str").str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S")</code>	Parse string to <code>Date</code> or <code>Datetime</code> type
strftime (format)	<code>pl.col("date").dt.strftime("%Y-%m-%d")</code>	Format datetime as string
date components	<code>pl.col("date").dt.year()</code> , <code>.dt.month()</code> , <code>.dt.day()</code>	Extract year, month, day components
time components	<code>pl.col("ts").dt.hour()</code> , <code>.dt.minute()</code> , <code>.dt.second()</code>	Extract hour, minute, second components

Method	Example	Description
date() / time()	<code>pl.col("ts").dt.date() , pl.col("ts").dt.time()</code>	Extract only the date or time part from a datetime
weekday	<code>pl.col("date").dt.weekday()</code>	Day of week (1=Monday, 7=Sunday)
ordinal_day	<code>pl.col("date").dt.ordinal_day()</code>	Day of year (1–366)
truncate	<code>pl.col("ts").dt.truncate("1h")</code>	- Round down to nearest time unit - e.g., "1d" , "1h" , "15m"
round	<code>pl.col("ts").dt.round("1d")</code>	Round to nearest time unit
offset_by	<code>pl.col("date").dt.offset_by("1mo")</code>	Calendar-aware date arithmetic: add "1mo" , "1y" , "7d" , etc.
date_range	<code>pl.date_range(pl.date(2024, 1, 1), pl.date(2024, 12, 31), interval="1d")</code>	Generate a series of dates at a regular interval
datetime_range	<code>pl.datetime_range(start, end, interval="1h", eager=True)</code>	- Generate datetime series - supports timezone-aware intervals
duration subtraction	<code>pl.col("end") - pl.col("start")</code>	Subtract two datetimes to get a Duration
total_seconds	<code>pl.col("dur").dt.total_seconds()</code>	Convert Duration to total seconds as integer
timestamp	<code>pl.col("ts").dt.timestamp("ms")</code>	Convert datetime to Unix timestamp in specified time unit

TABLE 16

List Operations

The `list` namespace enables vectorized operations over variable-length List columns, which is Polars' primary mechanism for nested data without exploding rows. This is significantly more memory-efficient than unnesting for most aggregation tasks.

Method	Example	Description
list.len	<code>pl.col("tags").list.len()</code>	Count elements in each list
list.sum / mean / min / max	<code>pl.col("scores").list.sum() , .list.mean()</code>	Aggregate all elements in each list
list.get	<code>pl.col("tags").list.get(0)</code>	Get element at index (negative index supported)
list.first / last	<code>pl.col("tags").list.first() , .list.last()</code>	Get first or last element of each list
list.contains	<code>pl.col("tags").list.contains("python")</code>	Boolean check if any element equals target value
list.sort	<code>pl.col("scores").list.sort()</code>	Sort each list ascending or descending
list.unique	<code>pl.col("tags").list.unique()</code>	Return unique elements from each list
list.slice	<code>pl.col("items").list.slice(1, 3)</code>	Extract sub-list from each row by offset and length
list.head / tail	<code>pl.col("items").list.head(2) , .list.tail(2)</code>	Take first or last N elements from each list
list.concat	<code>pl.col("list1").list.concat(pl.col("list2"))</code>	Concatenate two list columns element-wise
list.explode	<code>df.explode("tags")</code>	Expand list column into multiple rows (one per element)
list.gather	<code>pl.col("items").list.gather([0, 2])</code>	Select elements at specified indices from each list
list.sample	<code>pl.col("items").list.sample(n=2)</code>	Randomly sample N elements from each list

Method	Example	Description
list.eval	<code>pl.col("prices").list.eval(pl.element() * 1.1)</code>	Apply an expression to each list as if it were a miniature Series
list.set_union	<code>pl.col("a").list.set_union(pl.col("b"))</code>	Set union of two list columns element-wise
list.set_intersection	<code>pl.col("a").list.set_intersection(pl.col("b"))</code>	Set intersection of two list columns element-wise

TABLE 17

Null Handling

Polars uses Arrow's null bitmask — nulls are not stored as NaN or sentinel values, but as a separate validity mask. This means `null != NaN` ; `NaN` is a valid `Float64` value while `null` represents missing data of any type.

Method	Example	Description
is_null / is_not_null	<code>pl.col("a").is_null() , pl.col("a").is_not_null()</code>	Boolean mask for null or non-null values
fill_null (literal)	<code>pl.col("a").fill_null(0)</code>	Replace nulls with a literal value
fill_null (strategy)	<code>pl.col("a").fill_null(strategy="forward")</code>	Strategies: "forward" , "backward" , "mean" , "min" , "max" , "zero" , "one"
fill_null (expression)	<code>pl.col("a").fill_null(pl.col("b"))</code>	Fill nulls from another column
forward_fill	<code>pl.col("price").forward_fill()</code>	Propagate last valid value forward to fill nulls
backward_fill	<code>pl.col("price").backward_fill()</code>	Propagate next valid value backward to fill nulls
drop_nulls	<code>df.drop_nulls(subset=["col1", "col2"])</code>	Drop rows containing nulls (in any or specified columns)
null_count	<code>df.null_count()</code>	Count nulls per column across the full DataFrame
fill_nan	<code>pl.col("float_col").fill_nan(0.0)</code>	• Replace NaN values specifically — distinct from <code>fill_null</code> • affects float columns only
is_nan / is_not_nan	<code>pl.col("val").is_nan() , pl.col("val").is_not_nan()</code>	• Detect NaN in float columns • NaN is not null
coalesce	<code>pl.coalesce([pl.col("a"), pl.col("b"), pl.lit(0)])</code>	Return first non-null value across expressions (SQL-style COALESCE)

TABLE 18

Rolling and Time-Series Aggregations

Rolling window functions process a sliding window of rows, computing a statistic over each window position. Polars supports both row-count-based windows and time-duration-based windows via the `_by` variants.

Method	Example	Description
rolling_mean	<code>pl.col("price").rolling_mean(window_size=7)</code>	Moving average over a fixed number of rows
rolling_sum	<code>pl.col("sales").rolling_sum(window_size=30)</code>	Moving sum over a fixed number of rows
rolling_min / max	<code>pl.col("price").rolling_min(window_size=7) ,.rolling_max(window_size=7)</code>	Moving minimum and maximum
rolling_std / var	<code>pl.col("price").rolling_std(window_size=10)</code>	Moving standard deviation and variance

Method	Example	Description
rolling_mean_by (time)	<code>pl.col("value").rolling_mean_by("timestamp", window_size="2h")</code>	- Time-aware rolling mean - window defined by duration, not row count
rolling_sum_by (time)	<code>pl.col("sales").rolling_sum_by("date", window_size="7d")</code>	Time-aware rolling sum over a date/datetime column
ewm_mean	<code>pl.col("price").ewm_mean(span=20)</code>	- Exponentially weighted moving average <code>span</code> , <code>alpha</code> , or <code>halflife</code> parameter
ewm_std	<code>pl.col("price").ewm_std(span=20)</code>	Exponentially weighted standard deviation
group_by_dynamic	<code>df.group_by_dynamic("date", every="1w").agg(pl.col("sales").sum())</code>	Time-bucket aggregation: group by dynamic time windows
rolling_min_by / max_by	<code>pl.col("price").rolling_min_by("timestamp", window_size="1d")</code>	Time-aware rolling minimum and maximum

TABLE 19

Pivoting and Reshaping

Reshaping between wide and long formats is a fundamental preprocessing step. Polars' `unpivot()` (replacing deprecated `melt()`) converts wide to long, while `pivot()` goes long to wide — note that `pivot()` is an eager operation that requires schema knowledge upfront.

Method	Example	Description
unpivot (wide to long)	<code>df.unpivot(on=["jan", "feb", "mar"], index="product")</code>	- Reshape wide to long format <code>on</code> is the value columns, <code>index</code> is the id columns
pivot (long to wide)	<code>df.pivot(on="month", index="product", values="sales")</code>	- Reshape long to wide - column values become column headers
pivot aggregate	<code>df.pivot(on="month", index="product", values="sales", aggregate_function="sum")</code>	Aggregate duplicate key combinations during pivot
explode	<code>df.explode("tags")</code>	Expand List column to multiple rows — one row per element
unnest	<code>df.unnest("struct_col")</code>	Flatten Struct column into individual top-level columns
concat vertical	<code>pl.concat([df1, df2], how="vertical")</code>	- Stack DataFrames vertically - schemas must match
concat horizontal	<code>pl.concat([df1, df2], how="horizontal")</code>	Join DataFrames side-by-side by row position
concat diagonal	<code>pl.concat([df1, df2], how="diagonal")</code>	- Stack DataFrames with schema union - missing columns become null
transpose	<code>df.transpose(include_header=True, header_name="field")</code>	- Flip rows and columns - all values become the same type
melt (legacy)	<code>df.melt(id_vars=["id"], value_vars=["a", "b"])</code>	Deprecated since Polars 1.0 — use <code>unpivot(on=[...], index=[...])</code> instead

TABLE 20

Advanced Operations

These are powerful but less frequently used operations that enable complex data transformations. `map_elements` is an escape hatch for Python-level logic but should be avoided in performance-critical paths; prefer native expressions.

Method	Example	Description
when / then / otherwise	<code>pl.when(pl.col("age") > 18).then(pl.lit("adult")).otherwise(pl.lit("minor"))</code>	• Vectorized conditional logic • SQL CASE WHEN equivalent
Expr.replace	<code>pl.col("status").replace({"A": "Active", "I": "Inactive"})</code>	• Map values via dict • cleaner than <code>when/then</code> chains for simple lookups
Expr.replace_strict	<code>pl.col("code").replace_strict({1: "X", 2: "Y"}, default="other", return_dtype=pl.String)</code>	• Like <code>replace</code> but enforces complete mapping • unmapped values raise error unless <code>default</code> is set
join_where	<code>lf1.join_where(lf2, pl.col("a") < pl.col("b"), pl.col("c") == pl.col("d"))</code>	• Inequality and mixed-condition joins • use where equi-join is insufficient
gather (by indices)	<code>lf.gather([0, 5, 10])</code>	• Select rows by integer index array • lazy equivalent of row-position selection
map_elements	<code>pl.col("col").map_elements(lambda x: x * 2, return_dtype=pl.Int64)</code>	• Apply arbitrary Python function per-element • slow, avoid in hot paths
map_batches	<code>pl.col("col").map_batches(lambda s: custom_fn(s))</code>	• Apply a function to the whole Series at once • faster than <code>map_elements</code>
struct creation	<code>pl.struct(["col1", "col2"]).alias("nested")</code>	Combine multiple columns into a single Struct column
struct field access	<code>pl.col("struct_col").struct.field("fieldname")</code>	Access a named field inside a Struct column
Sample rows	<code>df.sample(n=100, seed=42)</code>	Randomly sample N rows
apply (group-wise)	<code>df.group_by("grp").map_groups(fn)</code>	• Apply a Python function to each group as a DataFrame • slow but flexible
unique combinations	<code>df.unique(subset=["col1", "col2"], keep="first")</code>	• Deduplicate rows by subset • <code>keep</code> can be <code>"first"</code>, <code>"last"</code>, or <code>"any"</code>

TABLE 21

Statistical Functions

Statistical expressions work in any Polars context and compose with group-by, window functions, and filters. `describe()` is useful for quick EDA; for production pipelines, use individual aggregation expressions to avoid collecting everything into Python.

Function	Example	Description
describe	<code>df.describe()</code>	Summary statistics (count, mean, std, min, max, percentiles) for all columns
corr	<code>pl.pearson_corr("col1", "col2")</code>	Pearson correlation coefficient between two columns
spearman_rank_corr	<code>pl.spearman_rank_corr("col1", "col2")</code>	• Spearman rank correlation • more robust than Pearson for non-linear data
quantile	<code>pl.col("age").quantile(0.9, interpolation="linear")</code>	Percentile with interpolation methods: <code>"linear"</code> , <code>"nearest"</code> , <code>"lower"</code> , <code>"higher"</code> , <code>"midpoint"</code>
std / var	<code>pl.col("price").std(ddof=1), .var(ddof=1)</code>	• Standard deviation and variance • <code>ddof=1</code> for sample, <code>ddof=0</code> for population
skew	<code>pl.col("price").skew()</code>	Measure of asymmetry in distribution

Function	Example	Description
kurtosis	<code>pl.col("price").kurtosis()</code>	Measure of tail heaviness (excess kurtosis by default)
entropy	<code>pl.col("probs").entropy(base=2)</code>	- Shannon entropy <code>base=2</code> for bits, <code>base=math.e</code> for nats
value_counts	<code>df["category"].value_counts(sort=True)</code>	Frequency table with optional sort by count
arg_sort	<code>pl.col("price").arg_sort()</code>	Return row indices that would sort the column
arg_max / arg_min	<code>pl.col("price").arg_max()</code> , <code>.arg_min()</code>	Index of the maximum or minimum value
n_unique	<code>pl.col("category").n_unique()</code>	Count of distinct values
approx_n_unique	<code>pl.col("user_id").approx_n_unique()</code>	- Fast approximate distinct count using HyperLogLog - much faster for large columns
mode	<code>pl.col("category").mode()</code>	Most frequently occurring value(s)

TABLE 22

Column Selectors

The `cs` module (`polars.selectors`) provides semantic column selection patterns — far more readable than manual type checks or regex filters on column names. Selectors compose with set operators to build complex selection logic in one expression.

Selector	Example	Description
cs.numeric()	<code>df.select(cs.numeric())</code>	Select all numeric columns (integer and float types)
cs.string()	<code>df.select(cs.string())</code>	Select all String/Utf8 columns
cs.boolean()	<code>df.select(cs.boolean())</code>	Select all Boolean columns
cs.temporal()	<code>df.select(cs.temporal())</code>	Select all temporal columns (Date, Datetime, Duration, Time)
cs.by_dtype()	<code>df.select(cs.by_dtype(pl.Float64, pl.Float32))</code>	Select columns matching specified dtype(s)
cs.by_name()	<code>df.select(cs.by_name("a", "b", "c"))</code>	Select columns by exact name
cs.starts_with()	<code>df.select(cs.starts_with("sales_"))</code>	Select columns whose names start with a prefix
cs.ends_with()	<code>df.select(cs.ends_with("_id"))</code>	Select columns whose names end with a suffix
cs.contains()	<code>df.select(cs.contains("2024"))</code>	Select columns whose names contain a substring
cs.matches()	<code>df.select(cs.matches(r"^[1-4]_"))</code>	Select columns whose names match a regex pattern
cs.all()	<code>df.select(cs.all())</code>	Select all columns (equivalent to <code>pl.col("*")</code>)
cs.first() / last()	<code>df.select(cs.first())</code> , <code>df.select(cs.last())</code>	Select only the first or last column
Selector union	<code>df.select(cs.numeric() cs.boolean())</code>	• Combine selectors with set union (<code> </code>)
Selector intersection	<code>df.select(cs.numeric() & cs.starts_with("q"))</code>	Columns matching both selectors using <code>&</code>
Selector negation	<code>df.select(~cs.numeric())</code>	Select all columns except those matching the selector
Selector difference	<code>df.select(cs.numeric() - cs.by_name("id"))</code>	Numeric columns excluding the <code>id</code> column
cs.expand_selector()	<code>cs.expand_selector(df, cs.numeric())</code>	Resolve selector to an explicit list of column names

TABLE 23

Practical Patterns and Examples

These are complete, idiomatic multi-step patterns that combine Polars primitives for common real-world tasks. Each pattern follows the principle: push all work into a single lazy query, optimize it, and materialize only once.

Technique	Example	Description
End-to-end streaming pipeline	<code>pl.scan_parquet("huge/*.parquet").filter(pl.col("year") == 2024).select("id", "sales").sink_parquet("filtered.parquet")</code>	Full scan → filter → project → sink pipeline without loading into RAM
GPU-accelerated pipeline	<code>pl.scan_parquet("data.parquet").group_by("key").agg(pl.col("val").sum()).collect(engine="gpu")</code>	• Execute aggregation on NVIDIA GPU via cuDF • auto-falls back to CPU on failure
Multi-LazyFrame CSE	<code>pl.collect_all([lf_sales, lf_returns, lf_inventory])</code>	Execute multiple related queries sharing common subexpressions in one pass
Conditional column creation	<code>df.with_columns(pl.when(pl.col("score") > 90).then("A").when(pl.col("score") > 80).then("B").otherwise("C").alias("grade"))</code>	Multi-branch conditional assignment with chained <code>when/then</code>
Dict-based column mapping	<code>df.with_columns(pl.col("code").replace({"A": "Active", "D": "Deleted"}).alias("status"))</code>	• Replace lookup with a dict • cleaner and faster than chained <code>when/then</code>
Type-safe aggregation pipeline	<code>(pl.scan_csv("sales.csv").cast({"amount": pl.Float64}).group_by("region").agg(pl.col("amount").sum(), pl.len().alias("n")).sort("amount", descending=True).collect())</code>	Cast on scan → <code>group_by</code> → multi-agg → sort → collect
Explode + aggregate list column	<code>df.explode("tags").group_by("tags").agg(pl.len().alias("count")).sort("count", descending=True)</code>	Flatten list column to rows, then count frequencies
Hive-partitioned write	<code>lf.sink_parquet(pl.PartitionBy("./output/", key="year"), mkdir=True)</code>	Write year-partitioned Parquet for faster downstream filtered scans
Profile a query	<code>df_result, timing = lf.profile()</code>	Execute and return both result and per-node timing breakdown for optimization
Pandas interop with zero copy	<code>arrow = df.to_arrow(); pandas_df = arrow.to_pandas(zero_copy_only=True)</code>	Convert via Arrow to pandas without data copy where possible
Join + enrich pattern	<code>(orders.join(customers, on="customer_id", how="left").with_columns(pl.col("revenue").fill_null(0).alias("revenue_clean")))</code>	Left join to enrich then clean nulls from missing matches
Cross join (cartesian)	<code>df_a.join(df_b, how="cross")</code>	• Produce all row combinations • use <code>.filter()</code> immediately after to avoid explosion
Rolling z-score per group	<code>df.with_columns(((pl.col("val") - pl.col("val").mean().over("group")) / pl.col("val").std().over("group")).alias("z_score"))</code>	Normalize within each partition using <code>over()</code>
Read multiple DB tables	<code>df = pl.read_database("SELECT a.*, b.cat FROM a JOIN b ON a.id=b.id", conn)</code>	• Push SQL join to the database • retrieve pre-joined result into Polars

References

Official Documentation

1. [Polars Python API Reference](#)
2. [Polars User Guide](#)
3. [Polars LazyFrame API](#)
4. [Polars DataFrame API](#)
5. [Polars Expressions API](#)
6. [Polars Data Types Reference](#)
7. [Polars Column Selectors](#)
8. [Polars Lazy API User Guide](#)
9. [Polars Streaming Guide](#)
10. [Polars GPU Support](#)
11. [Polars Query Optimization Guide](#)
12. [Polars Joins User Guide](#)
13. [Polars Aggregation Guide](#)
14. [Polars I/O Overview](#)
15. [Polars Arrow Interoperability](#)
16. [Polars Multiprocessing Guide](#)
17. [pl.scan_csv API](#)
18. [pl.scan_parquet API](#)
19. [pl.scan_ndjson API](#)
20. [pl.scan_ipc API](#)
21. [pl.read_csv API](#)
22. [pl.read_parquet API](#)
23. [pl.read_excel API](#)
24. [pl.read_database API](#)
25. [pl.read_database_uri API](#)
26. [pl.read_json API](#)
27. [pl.read_ndjson API](#)
28. [pl.read_ipc API](#)
29. [DataFrame.write_parquet API](#)
30. [DataFrame.write_excel API](#)
31. [LazyFrame.sink_parquet API](#)
32. [LazyFrame.sink_csv API](#)
33. [LazyFrame.sink_ndjson API](#)
34. [LazyFrame.sink_ipc API](#)
35. [LazyFrame.collect API](#)
36. [LazyFrame.collect_batches API](#)
37. [pl.collect_all API](#)
38. [LazyFrame.profile API](#)
39. [LazyFrame.explain API](#)
40. [LazyFrame.join API](#)
41. [LazyFrame.join_where API](#)
42. [LazyFrame.gather API](#)
43. [LazyFrame.gather_every API](#)
44. [DataFrame.group_by API](#)
45. [DataFrame.group_by_dynamic API](#)
46. [DataFrame.join API](#)
47. [DataFrame.filter API](#)
48. [DataFrame.with_columns API](#)
49. [DataFrame.select API](#)
50. [DataFrame.sort API](#)
51. [DataFrame.unique API](#)
52. [DataFrame.unpivot API](#)
53. [DataFrame.pivot API](#)
54. [DataFrame.explode API](#)
55. [DataFrame.unnest API](#)
56. [DataFrame.transpose API](#)
57. [DataFrame.rename API](#)
58. [DataFrame.drop API](#)
59. [DataFrame.sample API](#)
60. [DataFrame.describe API](#)
61. [DataFrame.rechunk API](#)
62. [DataFrame.estimated_size API](#)
63. [DataFrame.null_count API](#)
64. [DataFrame.drop_nulls API](#)
65. [DataFrame.with_row_index API](#)
66. [Expr.over API](#)
67. [Expr.cast API](#)
68. [Expr.alias API](#)
69. [Expr.filter API](#)
70. [Expr.sort_by API](#)
71. [Expr.fill_null API](#)
72. [Expr.fill_nan API](#)
73. [Expr.is_null API](#)
74. [Expr.is_nan API](#)
75. [Expr.forward_fill API](#)
76. [Expr.backward_fill API](#)
77. [Expr.replace API](#)
78. [Expr.replace_strict API](#)
79. [Expr.shift API](#)
80. [Expr.diff API](#)
81. [Expr.pct_change API](#)
82. [Expr.rank API](#)
83. [Expr.arg_sort API](#)
84. [Expr.rolling_mean API](#)
85. [Expr.rolling_sum API](#)
86. [Expr.rolling_mean_by API](#)
87. [Expr.rolling_sum_by API](#)
88. [Expr.rolling_min_by API](#)
89. [Expr.ewm_mean API](#)
90. [Expr.ewm_std API](#)
91. [Expr.cum_sum API](#)
92. [Expr.cum_min API](#)
93. [Expr.quantile API](#)
94. [Expr.std API](#)
95. [Expr.skew API](#)
96. [Expr.kurtosis API](#)
97. [Expr.entropy API](#)
98. [Expr.approx_n_unique API](#)
99. [Expr.n_unique API](#)
100. [Expr.pearson_corr API](#)
101. [Expr.spearman_rank_corr API](#)
102. [Expr.map_elements API](#)
103. [Expr.map_batches API](#)
104. [Expr.implode API](#)
105. [Expr.struct.field API](#)
106. [Expr.str.contains API](#)
107. [Expr.str.starts_with API](#)
108. [Expr.str.replace API](#)
109. [Expr.str.replace_many API](#)
110. [Expr.str.replace_all API](#)
111. [Expr.str.split API](#)
112. [Expr.str.extract API](#)

- 113. [Expr.str.extract_all API](#)
- 115. [Expr.str.slice API](#)
- 117. [Expr.str.to_integer API](#)
- 119. [Expr.str.strip_chars API](#)
- 121. [Expr.dt.hour API](#)
- 123. [Expr.dt.strftime API](#)
- 125. [Expr.dt.round API](#)
- 127. [Expr.dt.weekday API](#)
- 129. [Expr.dt.timestamp API](#)
- 131. [Expr.list.get API](#)
- 133. [Expr.list.sort API](#)
- 135. [Expr.list.gather API](#)
- 137. [Expr.list.set_union API](#)
- 139. [pl.len API](#)
- 141. [pl.concat_str API](#)
- 143. [pl.struct API](#)
- 145. [pl.date_range API](#)
- 147. [pl.from_pandas API](#)
- 149. [pl.enable_string_cache API](#)
- 151. [pl.Categorical type reference](#)
- 153. [pl.Array type reference](#)
- 155. [Polars Changelog / Releases](#)

Technical Blogs & Tutorials

- 157. [Polars Blog: Announcing Polars 1.0](#)
- 159. [Polars Blog: GPU Support with RAPIDS cuDF](#)
- 161. [Towards Data Science: Polars vs Pandas — Which Should You Choose in 2025?](#)
- 163. [Medium: Polars 1.0 Breaking Changes — What You Need to Know](#)
- 165. [Ander Steele Blog: Polars Group By Aggregations Deep Dive](#)
- 167. [Kevin Kho: Getting Started with Polars Lazy API](#)
- 169. [stefanbschneider: Polars Cheat Sheet on GitHub](#)
- 171. [Stack Overflow Polars Tag](#)
- 173. [Towards Data Science: Polars vs DuckDB for Data Engineering](#)
- 175. [Level Up Coding: Polars — 10 Tips and Tricks for 2026](#)

GitHub Repositories & Code Examples

- 176. [pola-rs/polars GitHub Repository](#)
- 178. [pola-rs/polars — Release Notes](#)
- 180. [pola-rs/polars — Changelog](#)
- 182. [pola-rs/polars — Streaming Engine Source](#)
- 184. [machow/polars-cookbook](#)

Video Resources

- 186. [YouTube: Polars Tutorial 2025 — Full Crash Course \(Rob Mulla\)](#)
- 188. [YouTube: Polars Lazy API Deep Dive \(DataTalks.Club\)](#)
- 190. [YouTube: Polars Group By and Window Functions \(Khuyen Tran\)](#)
- 192. [YouTube: Polars — The Pandas Killer? Benchmarks 2026 \(Mikkel Jans\)](#)

Industry Best Practice Guides & Books

- 193. [O'Reilly: Data Engineering with Polars \(2025\)](#)
- 195. [Apache Arrow Columnar Format Specification](#)
- 197. [connectorx Documentation](#)
- 199. [Calamine Excel Reader Documentation](#)

- 114. [Expr.str.len_chars API](#)
- 116. [Expr.str.strptime API](#)
- 118. [Expr.str.to_uppercase API](#)
- 120. [Expr.dt.year API](#)
- 122. [Expr.dt.date API](#)
- 124. [Expr.dt.truncate API](#)
- 126. [Expr.dt.offset_by API](#)
- 128. [Expr.dt.total_seconds API](#)
- 130. [Expr.list.len API](#)
- 132. [Expr.list.contains API](#)
- 134. [Expr.list.eval API](#)
- 136. [Expr.list.sample API](#)
- 138. [pl.when API](#)
- 140. [pl.int_range API](#)
- 142. [pl.concat API](#)
- 144. [pl.coalesce API](#)
- 146. [pl.datetime_range API](#)
- 148. [pl.from_arrow API](#)
- 150. [pl.Decimal type reference](#)
- 152. [pl.Enum type reference](#)
- 154. [Series.value_counts API](#)
- 156. [Polars Understanding Data Types \(User Guide\)](#)

- 158. [Polars Blog: Polars Streaming — The Next Generation \(Dec 2025\)](#)
- 160. [Polars Blog: Versioning and Upgrade Guide](#)
- 162. [Real Python: Polars vs Pandas 2026](#)
- 164. [Practical Data Science: Using Polars GPU Engine with RAPIDS cuDF](#)
- 166. [Christophe Rangeon: Polars Window Functions Explained](#)
- 168. [Calmcode: Polars Video Tutorials](#)
- 170. [Pola.rs Community Forum](#)
- 172. [Polars Cookbook Examples](#)
- 174. [Data School: Polars Tutorial for Beginners](#)

- 177. [pola-rs/polars — Python package source](#)
- 179. [pola-rs/polars-book — User Guide Source](#)
- 181. [pola-rs/polars — GPU Integration Tests](#)
- 183. [jorgecarleitao/polars Examples](#)
- 185. [realpython/materials — Polars Code Samples](#)

- 187. [YouTube: Polars vs Pandas — Side by Side Comparison \(Python Programmer\)](#)
- 189. [YouTube: Polars GPU Support Demo \(NVIDIA cuDF\)](#)
- 191. [YouTube: Polars Streaming for Large Datasets \(Data With Mo\)](#)

- 194. [Polars Official Discord Community](#)
- 196. [RAPIDS cuDF Documentation](#)
- 198. [ADBC \(Arrow Database Connectivity\) Documentation](#)
- 200. [FastExcel Documentation \(Polars Excel engine\)](#)